



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS orders its priority queue purely by $g(n)$, the actual path cost accumulated so far from the start. It has no idea where the goal is, so it expands outward evenly in every direction, like ripples on water, and only "discovers" it's close to the goal once it actually gets there. A* orders its queue by $f(n) = g(n) + h(n)$, adding an estimate of the remaining distance to the goal on top of the cost so far. That extra $h(n)$ term means A* is directed rather than blind: given two frontier nodes with the same $g(n)$, it will always pop the one that's estimated to be closer to the goal first. In practice this means A* explores a narrow corridor of nodes pointed straight at the food instead of UCS's wide, uniform circle, which is exactly why A* needs far fewer node expansions to find the same optimal path.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the true remaining cost to the goal. On this grid every move costs exactly 1 and the agent can only step Up, Down, Left, or Right, so the absolute minimum number of moves possible between any two cells (ignoring walls) is $|x_1 - x_2| + |y_1 - y_2|$ — which is precisely the Manhattan distance formula. Walls can only ever make the real path longer than that, never shorter, so Manhattan distance is always a lower bound on the true cost. That's what makes it admissible. If the heuristic were not admissible (i.e. it sometimes overestimated the real cost), A* could be tricked into

believing a longer path is actually cheaper than a shorter one it hasn't reached yet, so it would pop and commit to a suboptimal node before ever expanding the truly optimal one. The algorithm would still terminate and return A^* path, but it would lose its optimality guarantee — the same failure mode that makes DFS's paths winding, except now A^* would also be lying to itself about which path is "best" while doing it.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan distance would stop being admissible under 8-way movement. With diagonal moves allowed, the agent could cut directly across a corner in a single step, so the true shortest path length between two diagonally-offset cells becomes much smaller than $|dx| + |dy|$ — for example, going from (0,0) to (3,3) only takes 3 diagonal moves, but Manhattan distance would estimate 6. Since the heuristic would now be overestimating the real cost, it would violate admissibility and A^* could return suboptimal paths, as explained in Q2. Euclidean distance is the right metric to switch to here: $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ is the true straight-line distance, which is always a valid lower bound regardless of whether movement is 4-way or 8-way, so it stays admissible in both cases.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Distance-to-nearest-food only tells A^* about the next single pellet and completely ignores how expensive it will be to reach everything else afterwards, so it can walk the agent into a "dead end" pocket of food that leaves it stranded far from the rest. A stronger heuristic would estimate the cost of visiting *all* remaining food, not just the nearest one. One practical option, borrowed from the classic Pacman "food heuristic" idea, is: $h(n) = \text{distance}(\text{agent}, \text{farthest remaining food}) + \text{minimum spanning tree (MST) cost over all remaining food positions (using Manhattan distance as the edge weight between pellets)}$. The idea is that any full tour has to cover a distance at least as large as the MST connecting all the food together, plus the agent still has to travel from its current position to reach that food cluster in the first place — using distance to the farthest pellet (rather than the nearest) as that

"entry cost" keeps the estimate a true lower bound, so it stays admissible while being far more informed than single-nearest-food distance. It's more expensive to compute per node than a single Manhattan distance call, but for a food-clearing task it steers the agent away from dead-end pellets and towards genuinely efficient full-clear routes.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING